Mobile Game Tracker

PLANO DE GERENCIAMENTO DE CONFIGURAÇÃO

Enfoque DevOps — Integração e Entrega Contínuas (CI/CD)

Versão 1.3

Campo	Descrição
Projeto	Mobile Game Tracker
Gerente de Projeto	Gustavo Floriano
Gerente/Responsável de GCS-DevOps	Gustavo Floriano
Fábrica de Software / Squad	Gustavo Floriano
Repositório principal	mobile-game-tracker

Histórico de Revisões

Data	Versão	Descrição	Autor
10/08/2026	1.0	Elaboração do documento com enfoque em práticas DevOps (CI/CD).	Luiz Camargo
22/08/2026	1.1	Preenchimento básico de informações sobre o projeto	Gustavo Floriano
06/09/2026	1.2	Estrutura de pastas do projeto final após versão entregue a disciplina	Gustavo Floriano
18/10/2026	1.3	Atualização do processo de Issues, branching, Pull Requests e CI; distinção entre práticas implementadas e planejadas. Novas funcionalidades.	Gustavo Floriano

Sumário

Sumário 1
1. Introdução 2
1.1 Finalidade 3
1.2 Escopo 3
Este Plano de Gestão e Configuração de Software tem como objetivo auxiliar os integrates da squad, responsável pelo desenvolvimento e operação do sistema desenvolvido durante as aulas de Desenvolvimento Mobile. 3
1.3 Definições, Acrônimos e Abreviações 3
1.4 Referências 3
1.5 Visão Geral 4
2. Cultura DevOps e Papéis na Gerência de Configuração 4
3. Ferramentas, Ambientes e Infraestrutura 4
3.1 Ferramentas do Pipeline 4
3.2 Ambientes e Infraestrutura 5
3.3 Estratégia de Branching e Versionamento 5
4. Identificação da Configuração 5
4.1 Itens de Configuração 6
4.2 Convenção de Nomenclatura 6
4.3 Estrutura do Repositório 6
5. Integração Contínua (CI) 7
5.1 Etapas do Pipeline de CI 7
5.2 Política de Pull Request e Revisão de Código 7
5.3 Critérios de Qualidade (Quality Gate) 7
6. Entrega e Implantação Contínua (CD) 8
6.1 Etapas do Pipeline de CD 8
6.2 Estratégias de Implantação 8
6.3 Rollback e Recuperação 8
7. Infraestrutura como Código (IaC) 8
7.1 Versionamento de Infraestrutura 9
7.2 Ambientes Efêmeros 9
8. Controle de Configuração e Mudança 9
8.1 Processo de Solicitação de Mudança 9
8.2 Comitê de Controle de Mudança (CCM) 9
9. Baselines e Releases 9
9.1 Geração Automática de Baseline 9
9.2 Notas de Versão (Release Notes) 9
10. Estimativa do Status de Configuração 9
10.1 Monitoramento e Observabilidade 10
10.2 Métricas DORA 10

10.3 Relatórios e Auditorias 10
11. Marcos 10
12. Treinamento e Recursos 10
13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros 10
14. Quadro-Resumo do Pipeline CI/CD 11

1. Introdução

Este Plano de Gerenciamento de Configuração (PGC) descreve as atividades de Gestão de Configuração de Software (GCS) que serão executadas ao longo do ciclo de vida do produto, com ênfase em práticas DevOps — em particular Integração Contínua (CI), Entrega/Implantação Contínua (CD), Infraestrutura como Código (IaC) e automação de pipelines. As atividades envolvem identificar os itens de configuração, manter sua integridade e controlar sistematicamente as mudanças, com o mínimo de burocracia possível e o máximo de automação.

1.1 Finalidade

Estabelecer um padrão a ser seguido pela equipe/indivíduo, garantindo rastreabilidade, versionamento e controle de mudanças do produto, apoiado por pipelines automatizados de build, teste, integração e implantação.

1.2 Escopo

O Mobile Game Tracker é um aplicativo desenvolvido em Flutter e Dart para organizar e acompanhar uma biblioteca pessoal de jogos. Atualmente, oferece telas de início, coleção e estatísticas, apresentando informações como gênero, plataforma, descrição, horas jogadas e avaliações.

Este plano estabelece os procedimentos de gerenciamento de configuração do projeto, abrangendo código-fonte, testes automatizados, arquivos de configuração e documentação. O versionamento é realizado com Git e GitHub, utilizando Issues para rastrear demandas, branches para isolar alterações e Pull Requests para revisão. A Integração Contínua (CI) está implementada por meio de GitHub Actions, enquanto a Entrega Contínua (CD) permanece planejada, incluindo a geração e disponibilização automatizada de artefatos Android, como APK e AAB.

As práticas são dimensionadas para um projeto acadêmico, priorizando rastreabilidade, validação e controle de mudanças. O plano poderá ser atualizado de acordo com a evolução do aplicativo e os requisitos da disciplina.

1.3 Definições, Acrônimos e Abreviações

Termo	Significado
GCS	Gestão de Configuração de Software
CI	Integração Contínua (Continuous Integration)
CD	Entrega/Implantação Contínua (Continuous Delivery / Deployment)
IaC	Infraestrutura como Código (Infrastructure as Code)
CCM	Comitê de Controle de Mudanças
Baseline	Conjunto de itens de configuração que atingiram um estado comprovado de estabilidade
Pipeline	Sequência automatizada de etapas de build, teste, análise e implantação
Trunk-based Development	Estratégia de branching em que os desenvolvedores integram na branch principal com alta frequência
SemVer	Versionamento Semântico (MAJOR.MINOR.PATCH) - 1.0.0.

Termo	Significado
DORA Metrics	Métricas de desempenho DevOps (Lead Time, Deployment Frequency, MTTR, Change Failure Rate)

1.4 Referências

- Documento base: Processos de Desenvolvimento de Software e a Integração com GCS.
- Template do Plano de Gerenciamento de Configuração (RUP 7.0, IBM) — adaptado.
- Arquivo de Plano de Projeto — N1_faixa_8.pdf
- Repositório do projeto no GitHub — mobile-game-tracker

1.5 Visão Geral

Diferente de processos tradicionais em que a GCS é predominantemente manual e concentrada em marcos formais, neste plano a GCS é tratada como parte constante do pipeline de entrega: As Issues organizam o trabalho; branches isolam mudanças; commits registram a evolução; Pull Requests permitem revisão e execução da CI; e o merge integra mudanças aprovadas à branch principal. A automação existente fornece evidências por meio do histórico do GitHub Actions. A publicação automática de artefatos e a implantação contínua não são consideradas parte do fluxo atual sem configuração específica.

2. Cultura DevOps e Papéis na Gerência de Configuração

A GCS é uma responsabilidade compartilhada. Em uma squad acadêmica de uma pessoa, diferentes papéis podem ser exercidos pelo mesmo responsável, sem eliminar a necessidade de registrar decisões e verificar as mudanças.

Papel	Responsabilidades
Gerente de Projeto / Product Owner	Prioriza o backlog, autoriza mudanças de maior impacto e acompanha indicadores de entrega (DORA Metrics).
Engenheiro DevOps / Release Engineer	Projeta e mantém os pipelines de CI/CD, define a estratégia de *branching* e versionamento, administra o registro de artefatos/imagens e a IaC.
Desenvolvedor(a)	Integra código com frequência na branch principal, escreve testes automatizados, revisa Pull Requests e é corresponsável pela qualidade do que é implantado.
SRE / Operações *Site Reliability Engineering*	Monitora ambientes de produção, define SLIs/SLOs, participa de postmortems e realiza rollback quando necessário. - *Service Level Indicators (SLIs) and Service Level Objectives (SLOs).*
QA / Automação de Testes	Mantém a suíte de testes automatizados (unitários, integração, aceitação) executada nos pipelines de CI e CD.
Segurança (DevSecOps)	Define políticas de análise estática (SAST), análise de dependências (SCA) e gestão de segredos integradas ao pipeline.
Comitê de Controle de Mudanças (CCM)	Atua apenas em mudanças de alto risco/impacto (ex.: mudanças em produção fora do fluxo padrão); mudanças de rotina seguem o fluxo automatizado de PR + pipeline.

3. Ferramentas, Ambientes e Infraestrutura

3.1 Ferramentas do Pipeline

Ferramentas sugeridas adotadas pelo projeto:

Categoria	Ferramenta(s) sugerida(s)	Finalidade
Controle de versão	Git + GitHub	Versionamento de código-fonte, IaC e pipelines como (build + testes verdes) código
Orquestração de CI/CD	GitHub Actions	Automação de build, testes, análise e implantação
Build	FLutter Build	Geração da versão com todas as dependências necessárias.
Registro de artefatos	GitHub	Armazenamento e versionamento de imagens e pacotes
Gestão de segredos	Github Repository Secrets	Armazenamento seguro de credenciais usadas nos pipelines
Rastreamento de tarefas	GitHub Issues	Vinculação de commits/PRs a itens de trabalho
Stack do Projeto	Flutter	Desenvolvimento FullStack da aplicação
Dependências	pubspec.yaml / pubspec.lock	Gerenciamento e versionamento das dependências

3.2 Ambientes e Infraestrutura

Os ambientes devem ser, sempre que possível, provisionados via Infraestrutura como Código, garantindo paridade entre eles e permitindo recriação rápida e rastreável.

Ambiente	Finalidade	Gatilho de promoção
Desenvolvimento local	O desenvolvimento é realizado localmente, utilizando Flutter e Dart, enquanto o GitHub centraliza o código-fonte e a automação dos processos de validação por meio do GitHub Actions.	Merge/push na branch principal de desenvolvimento (dev)
Execução da CI	Existente. Runners do GitHub Actions executam análise, testes e builds.	Abertura de PRs para merge em outras branches.
Homologação / Staging (Não definido)	Não aplicável no fluxo atual; ambiente não configurado	Aprovação de merge para a branch de desenvolvimento (dev). Atualmente não configurado.
Produção (Não definido)	Não aplicável no fluxo atual; ambiente não configurado	Aprovação de merge para a branch principal (main). Atualmente não configurado.

3.3 Estratégia de Branching e Versionamento

O projeto adota um fluxo baseado em branches de curta duração e Pull Requests, alinhado ao GitHub Flow. A branch principal é destinada à integração do código validado; alterações de funcionalidades são desenvolvidas separadamente.

- Branch principal: **main**, usada como referência de integração do projeto.

- Branch para ambiente de testes: **dev**, usada para testes pré-deploy de funcionalidades.
- Branches de funcionalidade: feat/<descrição-curta>, por exemplo feat/store-page.
- Correções podem utilizar fix/<descrição-curta>, conforme a convenção adotada.
- Atualização de documentação, com docs/<descrição-curta>

Versionamento: adoção de Versionamento Semântico (MAJOR.MINOR.PATCH) e Conventional Commits (feat, fix, chore, docs, refactor…).

Foi criada a primeira versão de tags via github para adoção do MAJOR.MINOR.PATCH, sendo a atual na branch principal, **v1.0.0**

4. Identificação da Configuração

4.1 Itens de Configuração

Além do código-fonte, em um contexto DevOps os seguintes artefatos também são itens de configuração formais, versionados no mesmo repositório (ou repositórios compartilhados):

- Código-fonte da aplicação e dos testes automatizados
- Workflows de CI, por exemplo arquivos YAML em **.github/workflows/.**
- Templates de Issues versionados em **.github/ISSUE_TEMPLATE.**
- Arquivos de configuração e dependências, incluindo **pubspec.yaml**, **pubspec.lock** e **analysis_options.yaml**.
- Documentação do projeto, incluindo README e este plano
- Arquivos de configuração por ambiente (variáveis, feature flags) — exceto chaves secretas, que devem ficar ambientes seguros.

4.2 Convenção de Nomenclatura

Item	Convenção	Exemplo
Branch de feature	feat/<descrição-curta>	feat/store-page
Branch de correção	fix/ <descrição-curta>	fix/store-layout
Branch de docs	docs/ <descrição-curta>	docs/update-readme
Commit	Conventional Commits: <tipo>(<escopo>): <descrição>	feat(store): adiciona estrutura inicial da loja
Tag de release	Versionamento Semântico vX.Y.Z	v2.3.0
Issue	Título objetivo e tipo identificável	feat: implementar página da Store

4.3 Estrutura do Repositório

mobile-game-tracker

```
├── android
├── assets
├── ios
├── lib
│   ├── app/
│   │   ├── app.dart
│   │   └── json_loader.dart
│   ├── models/
│   │   └── game.dart
│   ├── screens/
│   │   ├── collection.dart
│   │   ├── dashboard.dart
│   │   ├── game_details.dart
│   │   └── status.dart
│   ├── widgets/
│   │   ├── dashboard/
│   │   │   ├── card_info.dart
│   │   │   ├── dashboard_info.dart
│   │   │   └── game_suggestion.dart
│   │   ├── game_details/
│   │   │   ├── control_button.dart
│   │   │   └── count_info_card.dart
│   │   ├── shared/
│   │   │   ├── game_card.dart
│   │   │   └── main_text.dart
│   │   └── status/
│   │   │   ├── hours_chart.dart
│   │   │   └── status_card.dart
│   └── main.dart
├── test
├── web
├── .gitignore
├── .metadata
├── analysis_options.yaml
├── diceb.iml
├── pubspec.lock
├── pubspec.yaml
└── readme.md
```

5. Integração Contínua (CI)

A CI valida automaticamente mudanças submetidas ao repositório. O workflow deve ser acionado nos eventos configurados em seu arquivo YAML. No fluxo de desenvolvimento com Pull Requests, os resultados dos checks são utilizados para decidir se a alteração está pronta para integração.

5.1 Etapas do Pipeline de CI

O pipeline atual deve ser documentado conforme as etapas efetivamente presentes no workflow do GitHub Actions. Para o projeto Flutter, o conjunto mínimo de validações é:

1. Checkout do repositório.
2. Configuração da versão de Flutter definida pelo workflow.
3. Restauração das dependências com **flutter pub get**.
4. Análise estática com **flutter analyze**.
5. Execução dos testes automatizados com **flutter test.**

6. **Build da versão android, windows e web.**

Build de APK/AAB, cobertura mínima, testes de integração, SAST/SCA e publicação de artefatos não devem ser apresentados como etapas existentes se não estiverem implementados no workflow. Podem ser adicionados em versões futuras do pipeline.

5.2 Política de Pull Request e Revisão de Código

- Cada funcionalidade deve ser trabalhada em uma branch separada e proposta por Pull Request.
- O PR deve explicar a mudança e, quando aplicável, referenciar a Issue relacionada.
- O diff deve ser revisado antes do merge para verificar escopo, legibilidade e possíveis regressões.
- Os checks automatizados configurados devem terminar com sucesso antes da integração, conforme as regras do repositório.
- A proteção da branch principal e a exigência de aprovação de PRs para merge por outra pessoa dependem das regras efetivamente habilitadas no GitHub. Sendo assim, ao menos a revisão de pelo menos um colaborador é necessário para implementação da funcionalidade na **main**.

5.3 Critérios de Qualidade (Quality Gate)

- Nenhuma falha nos checks obrigatórios de CI.
- Ausência de erros reportados por flutter analyze, de acordo com o critério configurado no workflow.
- Testes automatizados concluídos com sucesso.
- Revisão do diff e confirmação de que a alteração corresponde ao escopo da Issue.
- A aprovação é parte do processo de revisão dos Pull Requests e deve ocorrer antes do merge, conforme a política de revisão adotada pela equipe.

6. Entrega e Implantação Contínua (CD)

A implantação contínua não faz parte do fluxo atualmente documentado como implementado. O projeto ainda pode evoluir para automatizar a geração e publicação de APK/AAB ou a criação de Releases, mas isso exige definir o artefato, o gatilho, o versionamento e os critérios de publicação. Seguindo template do projeto, essa poderia ser uma implementação possível de CD no projeto:

6.1 Etapas do Pipeline de CD

- Gerar APK/AAB em um workflow de build separado ou como etapa explicitamente configurada da CI.
- Armazenar os artefatos em um local apropriado, como GitHub Actions Artifacts ou GitHub Releases, identificando a versão e o commit de origem. A ferramenta será definida na implementação da automação de CD.
- Criar uma Release do GitHub a partir de uma tag ou publicação de versão, caso essa automação seja adotada.
- Executar validações antes da publicação e documentar o procedimento de recuperação em caso de artefato inválido.

6.2 Estratégias de Implantação

Estratégia	Descrição
Rolling update	Substituição gradual das instâncias antigas pelas novas, minimizando indisponibilidade.
Blue-Green	Duas versões (ambientes) coexistem; o tráfego é comutado de uma vez para a nova versão, permitindo rollback imediato.
Canary Release	A nova versão recebe uma pequena fração do tráfego real, sendo ampliada gradualmente conforme a confiança.
Feature Flags	Funcionalidades são implantadas desligadas e ativadas em produção de forma independente do deploy, reduzindo o risco de releases.

Estratégias como rolling update, blue-green e canary são mais pertinentes a serviços implantados em infraestrutura controlada. Não se aplicam automaticamente à distribuição de um aplicativo Flutter móvel. A estratégia de distribuição do aplicativo deverá ser definida quando houver um canal de publicação escolhido.

6.3 Rollback e Recuperação

Até existir um pipeline de publicação, a recuperação consiste em corrigir o código, reverter a alteração quando necessário e gerar novamente o artefato. Um processo de rollback automatizado só será documentado após sua Implementação.

7. Infraestrutura como Código (IaC)

Toda a infraestrutura provisionada para os ambientes de Desenvolvimento, Homologação e Produção deve ser descrita como código versionado, submetida ao mesmo fluxo de Pull Request e pipeline usado para a aplicação.

7.1 Versionamento de Infraestrutura

- Alterações de infraestrutura seguem o mesmo processo de PR + revisão + pipeline ("plan" automático em PR, "apply" após aprovação).
- O estado da infraestrutura (state file) é armazenado de forma centralizada e versionada, nunca em máquinas locais.

7.2 Ambientes Efêmeros

Sempre que viável, ambientes de revisão (review apps) devem ser provisionados automaticamente por Pull Request, para validação isolada, e destruídos automaticamente após o merge ou fechamento do PR.

8. Controle de Configuração e Mudança

8.1 Processo de Solicitação de Mudança

O fluxo padrão para uma mudança é:

Etapa	Atividade

Planejar	Criar ou atualizar uma Issue com descrição, escopo e prioridade.
Isolar	Criar branch de funcionalidade ou correção.
Implementar	Realizar a mudança e atualizar os testes necessários.
Propor	Abrir Pull Request e relacioná-lo à Issue quando aplicável.
Validar	Executar CI e revisar o diff.
Integrar	Realizar merge após cumprir os critérios de qualidade configurados.
Registrar	Manter o histórico de Issue, commits, PR e execução da CI como evidência rastreável.

8.2 Comitê de Controle de Mudança (CCM)

Atualmente, o projeto não conta com o comitê por conta do tamanho do escopo do projeto. Porém, caso o projeto se escale a ponto da necessidade de um squad próprio, poderia ser implementado da seguinte forma:
O CCM é acionado apenas em mudanças de alto impacto ou risco elevado, como alterações estruturais de infraestrutura, mudanças de esquema de banco de dados em produção, ou exceções ao fluxo padrão (ex.: hotfix emergencial). Em situações rotineiras, a aprovação por pares (revisão de PR) substitui o comitê formal.

CCM é composto pelo Gerente do Projeto e o responsável de DevOps.

O CCM deve ser acionado em quedas críticas, como da aplicação/banco, mudanças necessárias em banco de dados em produção e mudanças arquiteturais no projeto.

9. Baselines e Releases

9.1 Geração Automática de Baseline

As baselines do projeto serão identificadas por versões estáveis do código-fonte, utilizando tags Git associadas aos respectivos commits. A adoção de Versionamento Semântico (SemVer) permitirá identificar versões e distinguir alterações corretivas, incrementais e incompatíveis.

Atualmente, a identificação de versões por tags é utilizada no repositório. A criação automatizada de baselines vinculadas à publicação de artefatos ou Releases permanece planejada e dependerá da implementação do processo de CD.

9.2 Notas de Versão (Release Notes)

As notas de versão podem ser preparadas manualmente para releases publicadas. A geração automática a partir de Conventional Commits é uma possibilidade futura, não uma capacidade já existente.

10. Estimativa do Status de Configuração

10.1 Monitoramento e Observabilidade

No estado atual, as evidências primárias são o histórico do GitHub, Issues, Pull Requests e execuções do GitHub Actions. Monitoramento contínuo de produção, alertas e health checks dependem da existência de um ambiente implantado e não são considerados configurados.

10.2 Métricas DORA

Métrica	Descrição
Deployment Frequency	Frequência com que o código é implantado em produção.
Lead Time for Changes	Tempo entre o commit e a implantação em produção.
Change Failure Rate	Percentual de implantações que resultam em falha em produção.
Mean Time to Restore (MTTR)	Tempo médio para restaurar o serviço após uma falha.

10.3 Relatórios e Auditorias

O histórico de pipelines (análise, testes e builds) e os logs do sistema de versionamento constituem, por si, o principal insumo de auditoria de GCS, dispensando relatórios manuais na maior parte dos casos.

11. Marcos

- CI com GitHub Actions: implementada e em uso.
- Página Store: em andamento enquanto o PR não for integrado.
- Proteção da branch principal: verificar a configuração efetiva.
- Automação de builds de release e publicação de artefatos: planejada.
- IaC, staging e monitoramento operacional: fora do escopo atual.

Marco	Situação
Organizar o trabalho por Issues e padronizar solicitações de funcionalidade/bug.	Adotado no fluxo do projeto; manter templates sincronizados com os arquivos publicados.
Configurar CI com GitHub Actions.	Em uso no fluxo atual; verificar o workflow e manter documentadas apenas as etapas existentes.
Desenvolver a estrutura inicial da página Store em branch própria e abrir PR	Em andamento na data desta revisão; concluir após validação, revisão e merge.
Evoluir a Store por funcionalidades menores e rastreáveis.	Próximo passo após concluir o PR atual.
Configurar proteção da branch principal e regras de aprovação.	Verificar e configurar conforme disponibilidade de colaboradores e requisitos do repositório.

Automatizar build e publicação de artefatos/releases.	Planejado; não implementado como CD.
Adotar IaC, staging e monitoramento operacional.	Fora do fluxo atual; reavaliar se o escopo exigir

12. Treinamento e Recursos

- Git e GitHub: branches, commits, Issues, Pull Requests e resolução de conflitos.
- GitHub Actions: leitura e manutenção de workflows, análise dos logs e diagnóstico de falhas.
- Flutter/Dart: análise estática e testes automatizados.
- Boas práticas de revisão de código, rastreabilidade e manutenção de documentação.
- Builds de release, versionamento SemVer e automação de Releases como tópicos futuros.

13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros

As dependências do aplicativo são declaradas em pubspec.yaml e resolvidas com o lockfile pubspec.lock. Alterações de dependências devem ser intencionais, testadas e incluídas no fluxo de revisão por Pull Request.

- Manter o lockfile versionado para favorecer resoluções consistentes.
- Avaliar compatibilidade e necessidade antes de atualizar dependências.
- Análise automatizada de vulnerabilidades e atualizações por Dependabot/Renovate podem ser adotadas futuramente,

14. Quadro-Resumo do Pipeline CI/CD

Resumo do fluxo padrão, do commit à produção:

Etapa	Descrição	Gatilho
Planejamento	Issue descreve funcionalidade, bug ou tarefa e sua prioridade.	Antes do desenvolvimento.
Branch e implementação	Mudança isolada em branch feature/ ou fix/.	Issue selecionada.
CI	Executa as verificações presentes no workflow, incluindo análise e testes conforme configurado.	Push/PR, de acordo com o YAML.
Pull Request	Apresenta as alterações para revisão, relaciona a mudança à Issue quando aplicável e permite acompanhar os checks automatizados.	Abertura ou atualização do Pull Request, conforme os eventos definidos no workflow do GitHub Actions.

Etapa	Descrição	Gatilho
Merge	Integra a mudança após checks e demais regras ativas serem atendidos.	PR aprovado conforme política vigente.
CD / publicação	Build de release, publicação de artefatos ou criação automatizada de Release.	Planejado; gatilhos ainda não definidos.
Staging / produção	Implantação e monitoramento de ambientes.	Não configurados no fluxo atual.